

Course Code: **IDC 222**



Scientific Computing & Data Visualization

Reji J

School of Data Science

IISER Thiruvananthapuram

MultiSet



Multi Set

- Multiset is an associative container similar to the set, but it can store multiple elements with same value.
- In other words, **duplicate values** are **allowed**.

Set Properties

- **Associative:** Elements are referenced by their key and not by their absolute position in the container.
- **Sorted:** Elements are stored in a sorted manner.
- **Equivalent keys:** Multiple elements in the container can have equivalent keys.
- **Immutable:** The value of elements can't be modified after they have been stored in the multiset.



MultiSet Declaration

- To implement a multiset in C++, we must include the `<set>` header file in our program.

```
#include <set>
```

- We can declare a set in C++ using the following syntax:

Syntax:

```
multiset<data_type> multiset_name = {key1, key2, key3, ...};
```

- Here, `key1, key2, key3, ...` - **key/value** to be stored in the set.

Example:

```
// initialize set with elements
multiset<int> my_multiset1 = {5, 3, 8, 1, 3};

// create an empty set
multiset<int> my_multiset2;
```



Sample Program

```
#include <iostream>
#include <set>
using namespace std;

int main()
{
    multiset<int> my_multiset = {5, 3, 8, 1, 3};

    for(int val : my_multiset)
    {
        cout << val << " ";
    }

    return 0;
}
```

Output:

1 3 3 5 8

As you can see, the multiset returned the values in a sorted manner and included the **duplicate** instances of 3.



MultiMap



MultiMap

- **Multimap** is an associative container similar to map, but it can have **multiple elements with same keys**.

MultiMap Declaration

- To use **Multimaps**, we must **include** the **map** header file:
- We can declare a **multimap** using the following syntax:

```
#include <map>
```

Syntax:

```
multimap<key_type, value_type> multimap_name = {{key1, value1},{key2, value2}, ...};
```

Example:

```
multimap<int,string> lang = {{1, "C"},{2, "C++"}, {1, "Python"}, {2, "R"}};
```



Sample Program

```
#include <iostream>
#include <map>
using namespace std;

int main()
{
    multimap<int, string> lang = {{1, "C"}, {2, "C++"}, {1, "Python"}, {2, "R"}};

    for (auto i : lang)
    {
        cout << i.first << " " << i.second << endl;
    }
    return 0;
}
```

OUTPUT :

```
1 C
1 Python
2 C++
2 R
```



Array in STL



Array in STL

- `std::array` is a container class that encapsulates fixed size arrays.
- It is similar to the **C-style arrays** as it stores multiple values of similar type.
- `Array` is defined in the `<array>` header so we must **include** this header before we can use **array**.

```
#include <array>
```

Array Declaration

```
Syntax:      array <datatype, size> array_name;
```

```
Example:      array <int, 5> marks;
```

- Here is how we declare a **array** of size **5** that stores a list of marks.



Array Initialization

- We can initialize arrays in two ways.

```
Method 1: array<int, 5> numbers = {1, 2, 3, 4, 5};
```

```
Method 1: array<int, 5> marks {10, 20, 30, 40, 50};
```

- Here, we initialized two arrays named **numbers** and **marks** both of **size 5** that stores element of type **int**.
-



Access elements of array in STL

- **Method 1:** using the `[]` operator and **index** of the array element.

```
array<int, 5> marks = {10, 20, 30, 40, 50};  
  
// access first array element  
cout << marks[0]; // returns 10  
  
// access second array element  
cout << marks[2]; // returns 30
```

- **Note:** Accessing elements using `[]` operator doesn't check for **out of bound error**.
- An out of bounds error occurs when a program tries to access data outside the allowed range.
- **Method 2:** using `at()` method which checks for **out of bound error**.

```
cout << marks.at(0) // returns 10  
cout << marks.at(2); // returns 30  
cout << marks.at(10) // throws std::out_of_range exception
```



Modify elements of array in STL

- To modify an element at a particular index, we can again use `[]` and `at()`.

```
array<int, 5> marks = {50, 67, 88, 98, 34};  
  
// modify the 3rd element using []  
marks[2] = 76;  
  
// modify the first element using at()  
marks.at(0) = 90;
```



front() , back() and size()

```
#include <iostream>
#include <array>

using namespace std;

int main()
{
    array<int,6> a {11,22,33,44,55,66};
    cout << a.size(); // 6
    cout << a[0]; // 11
    cout << a[3]; // 44
    cout << a.front(); // 11
    cout << a.back(); // 66
    return 0;
}
```

- **front() and back() function:**

These methods are used to access the first and the last element of the array directly.

- **size() function:**

used to get the number of elements in the array



`fill()` and `empty()`

- **`fill()` function:**

This is specially used to initialize or fill all the indexes of the array with a similar value.

```
array <int, 5> marks;  
  
// fill the array with zeros  
marks.fill(0);  
cout << "Array after fill()" << endl;  
for(int m: marks)  
{  
    cout << m << " ";  
}
```

- **`empty()` function:**

This function is used to check whether the declared STL array is empty or not, if it is empty then it returns true else false.

```
array <int, 0> marks;  
  
if(marks.empty())  
{  
    cout << "The array is empty";  
}  
else  
{  
    cout << "The array is not empty";  
}
```



Sample Program

```
// Direct initialization
```

```
#include <iostream>
```

```
#include <array>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    array<int, 5> numbers {1, 2, 3, 4, 5};
```

```
    cout << "The elements are: " << endl;
```

```
    // for each loop to print the elements
```

```
    for(int num: numbers)
```

```
    {
```

```
        cout << num << " ";
```

```
    }
```

```
    cout << endl;
```

```
}
```

```
// Taking input from user
```

```
#include <iostream>
```

```
#include <array>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    array<int, 5> arr;
```

```
    cout << "Enter 5 integers: ";
```

```
    for (int i = 0; i < arr.size(); i++)
```

```
    {
```

```
        cin >> arr[i];
```

```
    }
```

```
    cout << "You entered: ";
```

```
    for (int i = 0; i < arr.size(); i++)
```

```
    {
```

```
        cout << arr[i] << " ";
```

```
    }
```

```
    return 0;
```

```
}
```




```
#include <iostream>
#include <array>
using namespace std;

int main()
{
    array<int, 5> arr = {10, 20, 30, 40, 50};
    cout << "First element: " << arr[0] << endl;
    cout << "Last element: " << arr[4] << endl;
    return 0;
}
```

```
cout << arr[1];
cout << arr.at(1);
cout << arr.front();
cout << arr.back();
```

```
arr[2] = 100;

cout << arr.size();

for (int i = 0; i < arr.size(); i++)
{
    cout << arr[i] << " ";
}

for (int x : arr) {
    cout << x << " ";
}
```



2D array

```
#include <iostream>
#include <array>

using namespace std;

int main()
{
    // Define a 3x3 2D array using std::array
    array<array<int, 3>, 3> matrix = {{{1, 2, 3}, {4, 5, 6}, {7, 8, 9}}};

    cout << "2D Array Elements:\n";
    for (auto row : matrix)
    {
        for (int num : row)
        {
            cout << num << " ";
        }
        cout << endl;
    }
    return 0;
}
```



2D array - user input

```
#include <iostream>
```

```
#include <array>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    //2x3 2D array
```

```
    array<array<int, 3>, 2> matrix;
```

```
    // Taking user input
```

```
    cout << "Enter 6 elements for the 2x3 matrix:\n";
```

```
    for (int i = 0; i < 2; i++)
```

```
    {
```

```
        for (int j = 0; j < 3; j++)
```

```
        {
```

```
            cin >> matrix[i][j];
```

```
        }
```

```
    }
```

```
    cout << "\n2D Array Elements:\n";
```

```
    for (auto row : matrix)
```

```
    {
```

```
        for (int num : row)
```

```
        {
```

```
            cout << num << " ";
```

```
        }
```

```
        cout << endl;
```

```
    }
```

```
    return 0;
```



2D array

```
#include <iostream>
using namespace std;
int main() {
    int rows = 3, cols = 3;
    int matrix[3][3]; // Fixed-size 2D array
    cout << "Enter 9 elements for the 3x3 matrix:\n";
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            cin >> matrix[i][j];
        }
    }
    // Display the matrix
    cout << "\nMatrix elements:\n";
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            cout << matrix[i][j] << " ";
        }
        cout << endl;
    }
    return 0;
}
```



STL Algorithm



Algorithm

- In C++, the Standard Template Library (STL) provides a rich set of **algorithms** that can be utilized to perform operations like **sorting**, **searching**, and **manipulating** elements of containers (like arrays, vectors, etc.).
- These algorithms can be accessed using the `<algorithm>` header.

```
#include <algorithm>
```

- The `<algorithm>` library in C++ offers many useful functions like.

`sort()` , `copy()` , `move()` , `swap()` etc . . .

- We will be using **vectors** to understand the working of all these functions.
- However, these functions work on other **STL containers** as well.



Commonly Used Algorithms

- The `<algorithm>` library in C++ offers many useful functions.

Functions	Description
<code>sort()</code>	Sort the elements of the container.
<code>copy()</code>	Copy elements within a given range.
<code>move()</code>	Move the given range of elements.
<code>swap()</code>	Exchange values of two objects.
<code>merge()</code>	Merge sorted ranges.
<code>replace()</code>	Replace the value of an element.
<code>remove()</code>	Remove an element.



sort()

- To sort a vector in ascending order, we can use the sort() function.
- These algorithms can be accessed using the **<algorithm>** header.

Syntax: `sort(first, last)`

first - iterator specifying the **beginning** of the sorting range (inclusive)

last - iterator specifying the **end** of the sorting range (exclusive)

```
int main()
{
    vector<int> vec = {44, 22, 33, 11, 55};

    //sort the elements of the vector
    sort(vec.begin(), vec.end());

    for(int num : vec)
    {
        cout << num << " ";
    }
    return 0;
}
```

OUTPUT:

11 22 33 44 55



copy()

- We use the copy() function to **copy a given range of elements from one vector to another.**

Syntax: `copy(first, last, result)`

first - iterator specifying the beginning of the range to copy (inclusive)

last - iterator specifying the end of the range to copy (exclusive)

result - iterator specifying the position in the destination vector where the elements will be copied.



```
#include <algorithm>
#include <vector>
#include <iostream>
using namespace std;

int main()
{
    vector<int> source = {11, 22, 33, 44, 55};
    vector<int> destination(5);

    cout << "Destination Vector before copying" << endl;
    for(int num : destination)
    {
        cout << num << " ";
    }
    cout << endl;

    // copy the contents of source to destination
    copy(source.begin(), source.end(), destination.begin());

    cout << "Destination Vector after copying" << endl;
    for(int num : destination)
    {
        cout << num << " ";
    }
    cout << endl;
    return 0;
}
```

OUTPUT:

Destination Vector before copying

0 0 0 0 0

Destination Vector after copying

11 22 33 44 55

```

#include <algorithm>
#include <vector>
#include <iostream>
using namespace std;

int main()
{
    vector<int> source = {11, 22, 33, 44, 55};

    // create destination vector of size 5
    vector<int> destination(5);

    cout << "Destination Vector before copying" << endl;
    for(int num : destination)
    {
        cout << num << " ";
    }
    cout << endl;

    // copy the contents of source to destination
    copy(source.begin(), source.end(), destination.begin());

    cout << "Destination Vector after copying" << endl;
    for(int num : destination)
    {
        cout << num << " ";
    }
    cout << endl;
    return 0;
}

```

OUTPUT:

Destination Vector before copying

0 0 0 0 0

Destination Vector after copying

11 22 33 44 55

move()

- We can use the move() function to move elements from one vector to another.

Syntax: `move(first, last, result)`

swap()

- We can swap the contents of two STL vectors using the swap() function.

Syntax: `swap(x , y)`

Here, **x** and **y** are containers whose contents need to be swapped.



move()

```
#include <algorithm>
#include <vector>
#include <iostream>
using namespace std;

int main()
{
    vector<string> source = {"apple", "banana", "cherry"};
    vector<string> destination(3);

    // perform the move operation
    move(source.begin(), source.end(), destination.begin());

    cout << "After move:" << endl;
    cout << "-----" << endl;
    cout << "source: ";
    for(string str : source)
    {
        cout << str << " ";
    }
    cout << endl;

    cout << "destination: ";
    for(string str : destination)
    {
        cout << str << " ";
    }
    cout << endl;
    return 0;
}
```

OUTPUT:

After move:

source:

destination: apple banana cherry

swap()

```
#include <algorithm>
#include <vector>
#include <iostream>
using namespace std;

int main()
{
    vector<int> vec1 = {1, 2, 3};
    vector<int> vec2 = {4, 5, 6};

    // swap vec1 and vec2
    swap(vec1, vec2);

    // print vec1 and vec2 after swap
    cout << "After swap:" << endl;

    cout << "vec1: ";
    for(int num : vec1)
    {
        cout << num << " ";
    }
    cout << endl;

    cout << "vec2: ";
    for(int num : vec2)
    {
        cout << num << " ";
    }
    cout << endl;
    return 0;
}
```

OUTPUT:

After swap:

vec1: 4 5 6

vec2: 1 2 3

